Reinforcement Learning — Homework Assignment
Deep Q-Networks (DQN) on Atari

Tal Fiskus
fiskust@biu.ac.il

Submission Deadline: 13/05/2026
Work can be done in pairs

Objective

The goal of this assignment is to implement a **DQN agent** in **PyTorch**.

As background for this assignment, you must read the original DQN paper (from 2013):

Playing Atari with Deep Reinforcement Learning

Task Description

You should implement the DQN agent from scratch in PyTorch and train it on **a single** Atari game chosen from the following spreadsheet:

Atari games spreadsheet

In that spreadsheet:

- Write your ids in the row of the game you chose, based on open space.
- I suggest refreshing the page before selecting the environment so you won't override each other.
- Some games are marked in **bold**. These are games you should probably try to avoid since they take a long time to converge with DQN, if ever...
- The sheet also includes a **"Minimum score to beat (Must)"** for each game, which is the random agent score - if you didn't pass this score, the grade will probably not be so good.
- The sheet also includes a **"Score to beat (Good grade)"** for each game, which is the best linear agent score of that time - this is a good grade reward to beat.
- The sheet also includes a **"Score to beat (Bonus Grade)"** for each game, which is the DQN score of that time - this is a bonus grade reward to beat. Push yourself within the assignment limits.

Implementation Requirements

1. Read the DQN paper and implement the method in **PyTorch only**.
2. **Do not use any RL library.**
3. **Use the Atari ALE API**
4. Use ALE version NoFrameskip-v4
5. Train your agent on **one Atari game** from the spreadsheet.
6. Your implementation must use the **same preprocessing as in the original paper** for Atari games.
7. You **can't use the exact hyperparameters** from the paper, play with them a little bit, and push yourself to the best.
8. **This is not a competition**, potentially you can all get the maximum grade, you only compete with yourself...

Training Requirements

1. You may train for an **unlimited number of training steps**.
   Here, a single call to `env.step(...)` is considered **one training step**.
2. During training, you must track performance over time:
   - Every **10,000 steps**, perform an evaluation.
   - In each evaluation, run one or more **evaluation episodes**.
   - Record the **total reward** obtained during evaluation.
3. In the literature, the standard practice is to evaluate over **100 episodes** and report the average score. You may use fewer evaluation episodes, in any case you **must clearly state in the report how many evaluation episodes you used, and how you evaluate the results**.
4. Consider which exploration-exploitation policy you should use to maximize your results in evaluation..
5. For the **final reported result**, you must train the agent 3 times and report the **final rewards for these 3 different training runs**.

Minimum Performance Expectation

Each game in the spreadsheet has a **minimum score to beat**, which is higher than the score of a random agent.

- If your agent does **not** pass this minimum score, your grade will be negatively affected.
- If you see that your chosen game is not working well, **switch to another game**.
- However, if your game is already working, there is **no reason to switch games**.

Your **effort will be considered in grading**, but do not continue to invest time in a game that clearly does not work without trying an alternative.

If you switch games, make sure to document:

- which game(s) did you try,
- what worked,
- what did not work,
- and why did you decide to switch.

Report Requirements

Your final submission must include a **detailed PDF report**. The report should describe your code, implementation choices, and results.

At minimum, the report must include:

1. The **game you chose**.
2. A description of the **preprocessing** you implemented.
3. The **hyperparameters** you used.
4. A description of what you tried that **worked**.
5. A description of what you tried that **did not work** (for example: failed hyperparameter choices, unsuccessful game choices, etc.).
6. 3 graphs showing the **learning progress** for each training run: specifically, the evaluation rewards as a function of training time, with one point every **10k training steps**, until the training limit you chose.
7. The number of **evaluation episodes** used in each evaluation.
8. The number of **training steps** used in each training.
9. The **final reward**, reported as a list of **3 independent training runs**, and the average of these runs as a single scalar.

Submission Format

You must submit:

- a **PDF report**, and
- a **ZIP file** containing your Python code.

Collaboration Policy

You may work **individually or in pairs**.

Summary Checklist

- Read the DQN paper.
- Implement DQN in PyTorch from scratch.
- Use one Atari game from the provided spreadsheet.
- Follow the original Atari preprocessing.
- Evaluate every 10k steps.
- Plot evaluation reward versus training steps.
- Train 3 runs for a single game.
- Report the final result as an average over 3 runs.
- Submit a PDF report and a ZIP file with code.

Good Luck!